

Supplier Risk Analytics — ANOVA-Based Supplier Risk Flagging

Statistically identify which suppliers in a sourcing base are genuinely under-performing on **quality**, **delivery**, and **cost** — not just which ones *look* worse on a raw average — using One-Way ANOVA and Tukey's HSD post-hoc test, with results exportable to Power BI for team reporting.

Why this exists

Procurement teams sit on years of order-history data but rarely have a statistically defensible way to answer one question: "*Which suppliers are actually worse than the rest — or is it just noise?*" Sorting 20+ suppliers by their raw average defect rate or lead-time variance risks acting on random sampling noise. This project runs a formal statistical test first, and only names individual suppliers once that test confirms a real difference exists.

How it works

Step	What happens	Why
1. Load & validate	Reads order-history CSV, drops nulls/negatives/duplicates	Real data isn't clean by default — never silently impute a defect rate
2. One-Way ANOVA	Tests each metric (<code>scipy.stats.f_oneway</code>) across all suppliers	Confirms supplier identity is a real driver of the metric, not chance, <i>before</i> anyone is named
3. Tukey HSD post-hoc	Pairwise comparison across all suppliers (<code>statsmodels.stats.multicomp.pairwise_tukeyhsd</code>)	Only runs if ANOVA is significant; controls the false-positive rate that 190 raw t-tests would otherwise produce
4. Risk tiering	Counts how many peers each supplier is confirmed worse than, maps to CRITICAL / HIGH / WATCH / OK	Turns a p-value into a sourcing decision

Significance threshold: $\alpha = 0.05$ (industry-standard convention for quality/manufacturing statistics). Tested at $\alpha = 0.01\text{--}0.20$ on the sample dataset — see `docs/` — with no change in which suppliers were flagged, since the injected effect sizes are large; on real, messier data alpha choice will matter more.

Key results (sample dataset: 1,000 orders, 20 suppliers)

Risk category	Metric	F-statistic	p-value
Quality	<code>Defect_Rate_PPM</code>	759.75	< 0.000001
Delivery	<code>Lead_Time_Variance_Days</code>	233.71	< 0.000001
Cost	<code>Price_Volatility_Pct</code>	275.04	< 0.000001

All three came back statistically significant. Tukey HSD isolated **4 of 20 suppliers** as confirmed outliers, one of which (multi-risk) was flagged in two categories simultaneously.

Repository structure

```

└─
supplier_risk_flat.py      # Main script: load → ANOVA → Tukey → flag (start here)
supplier_risk_from_csv.py # Fuller version with a dedicated data-validation layer
visualize_before_after.py # Raw order-level scatter vs. per-supplier average, per metric
supplier_risk_analysis.pptx # Slide deck: methodology, results, decision framework, Power BI rollout
supplier_order_history.csv # Sample/synthetic order-history dataset
```

Getting started

```

└─
bash
pip install pandas scipy statsmodels matplotlib
python supplier_risk_flat.py
```

Point `CSV_PATH` at the top of the script to your own order-history export. Required columns: `Supplier`, `SKU_ID`, `SKU_Category`, `Defect_Rate_PPM`, `Lead_Time_Variance_Days`, `Price_Volatility_Pct`.

Sample output



Loaded 1000 rows across 20 suppliers

--- Defect_Rate_PPM ---

ANOVA: F = 759.75, p = 0.000000

Flagged suppliers:

Volt Precision Parts — mean 317.86, worse than 19/19 suppliers

Apex Drivetrain Solutions — mean 183.17, worse than 18/19 suppliers

Decision framework

Tier	Threshold	Action
CRITICAL	≥60% of peers	Block new purchase orders
HIGH	30–59% of peers	Restrict volume, qualify a backup supplier
WATCH	1–29% of peers	Monitor, no order restriction yet
OK	0% of peers	No statistically confirmed issue

Bringing it to Power BI

Export the results (CSV / Excel) to shared storage → Power BI "Get Data" → build KPI cards, a supplier × risk-category heatmap, and a bad-actor leaderboard → publish to a workspace shared with the sourcing team. Full walkthrough in the accompanying slide deck.

Disclaimer

The included dataset is synthetic, generated with known injected outliers to validate the statistical pipeline end-to-end. Swap in a real ERP export via `CSV_PATH` before using this for an actual sourcing decision.